

Software Development Life Cycle (SDLC) Model Report

E-Commerce Web Application

Course: Software Engineering Lab

Version: 1.0

Group Members:

- Iftekhar Sourav — 23524202107
- Musfiqur Rahman Sama — 23524202117
- Towfiq Omar Rakin — 23524202131
- Zarin Tasnim Rahman Tuly — 23524202139

1. Introduction

The **Software Development Life Cycle (SDLC)** is a structured process that defines the stages a software project passes through — from initial planning to deployment and maintenance. Selecting the right SDLC model is critical to aligning team size, timeline, requirement stability, and project complexity.

This report justifies, describes, and applies the **Agile (Scrum-based Iterative)** SDLC model to the E-Commerce Web Application, and presents a sprint-by-sprint execution plan grounded in the SRS v1.0 functional requirements.

2. SDLC Model Selection

2.1 Candidate Models Considered

Model	Pros	Cons	Fit for This Project
Waterfall	Simple, well-documented phases	Inflexible; late testing; poor for changing requirements	❌ Too rigid for a learning-oriented lab project
Spiral	Risk-driven, good for large systems	Costly; complex management overhead	❌ Overkill for a small academic team
V-Model	Strong verification at each stage	Still sequential; limited flexibility	❌ Lacks iterative feedback loops
Agile (Scrum)	Iterative, adaptive, team-friendly, fast delivery	Requires active collaboration	✅ Best fit — explained below
Incremental	Builds in modules; early partial delivery	Less adaptive than full Agile	✅ Partial fit; Agile subsumes its benefits

2.2 Recommended Model: Agile (Scrum-Based Iterative)

Agile with Scrum is selected as the primary SDLC model for the following reasons:

Justification Factor	Rationale
Small team (4 members)	Scrum is designed for teams of 3–9; daily standups and sprint reviews are lightweight and practical
Academic timeline	Fixed semester deadline maps naturally to time-boxed sprints
Well-defined but expandable scope	Core features are clear (from SRS), but future enhancements (payments, reviews, wishlists) allow iterative extension
Django + web stack	Modular Django apps (users, products, cart, orders) align perfectly with sprint-based feature delivery
Early feedback loops	Each sprint produces a working increment, enabling early testing and peer review
Risk mitigation	Defects found early in short cycles rather than discovered only at final delivery

3. Agile Scrum Overview

Scrum is a lightweight Agile framework that organizes work into **Sprints** — time-boxed iterations (typically 1–2 weeks). Each sprint produces a potentially shippable product increment.

3.1 Scrum Roles (Adapted for Academic Context)

Scrum Role	Team Member(s)	Responsibility
Product Owner	Towfiq Omar Rakin	Maintains Product Backlog; prioritizes features per SRS
Scrum Master	Musfiqur Rahman Sama	Facilitates sprints; removes blockers; tracks progress
Development Team	All 4 members	Designs, codes, tests, and documents each sprint increment

3.2 Scrum Ceremonies

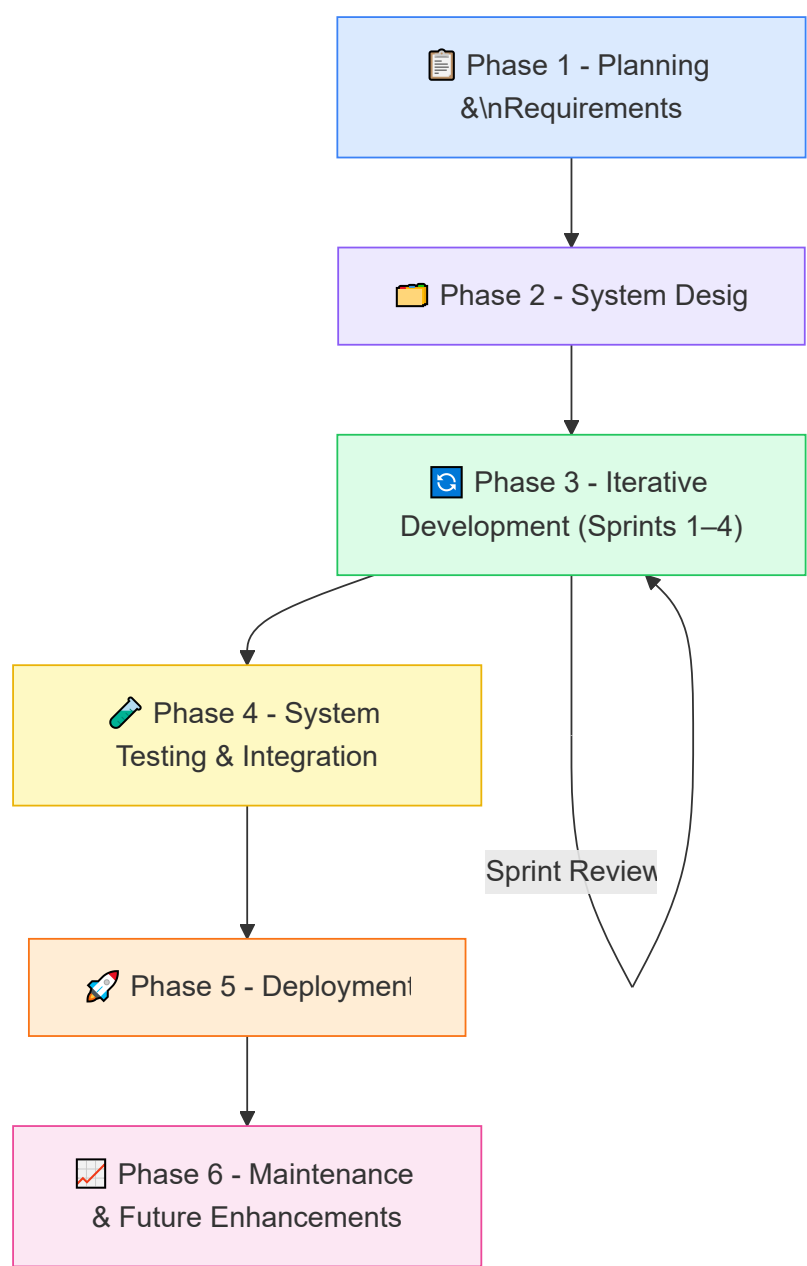
Ceremony	Frequency	Duration	Purpose
Sprint Planning	Start of each sprint	1 hour	Define sprint goal and select backlog items
Daily Standup	Every working day	15 minutes	Sync progress; identify blockers
Sprint Review	End of each sprint	30–45 min	Demo working increment to team/instructor
Sprint Retrospective	After Sprint Review	30 min	Reflect on process; improve next sprint

3.3 Scrum Artifacts

Artifact	Description
Product Backlog	All features derived from SRS FR1–FR28, prioritized by value
Sprint Backlog	Subset of Product Backlog committed to in current sprint
Increment	Working, tested software delivered at end of each sprint
Definition of Done	Feature is coded, unit tested, documented, and peer-reviewed

4. SDLC Phase Breakdown

Phase Overview Diagram



4.1 Phase 1 — Planning & Requirements

Duration: Week 1

Deliverables: SRS Document (completed), Project Plan, Product Backlog

Activities:

- Elicit and document all functional and non-functional requirements
- Define user classes (Guest, Registered User, Admin)
- Prioritize features into Product Backlog items (PBIs)
- Estimate story points for each PBI
- Set up development environment (Python, Django, SQLite, Git repository)

Product Backlog (derived from SRS):

Priority	Feature	SRS References	Story Points
P1	User Registration & Login	FR1–FR8	5
P1	Product Listing & Detail View	FR13–FR16	3
P1	Shopping Cart (Add/Remove/Update)	FR17–FR21	8
P1	Checkout & Order Creation	FR22–FR25	8
P2	Order History & Detail	FR26–FR28	5
P2	Admin Product Management	FR9–FR12	8
P3	Responsive UI & Navigation	NFR (Usability)	5
P3	Security (CSRF, Hashing, Sessions)	NFR (Security)	5

4.2 Phase 2 — System Design

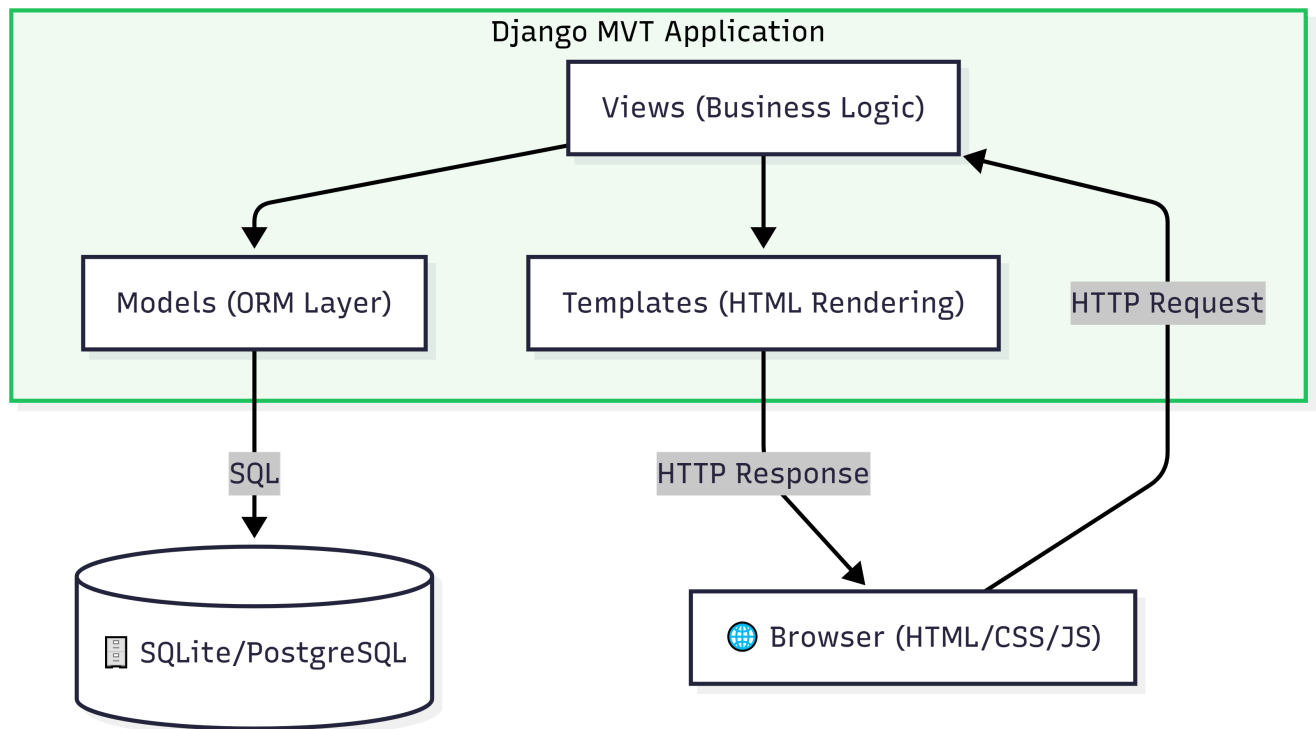
Duration: Week 2

Deliverables: Architecture Diagram, Database Schema, UI Wireframes

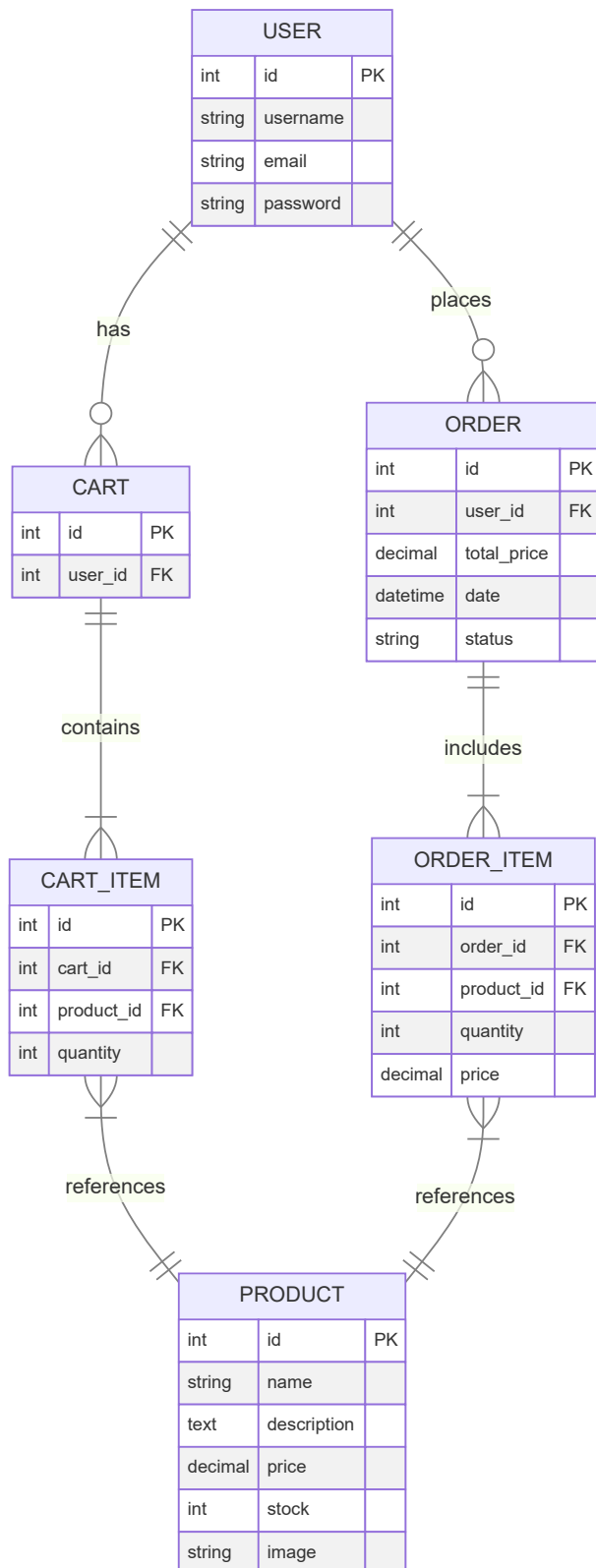
Activities:

- Define Django MVT architecture (Models, Views, Templates)
- Design relational database schema (User, Product, Cart, CartItem, Order, OrderItem)
- Create URL routing plan
- Design UI wireframes for all major pages
- Define REST-like endpoints for Django views

Architecture:



Database Schema:



4.3 Phase 3 — Iterative Development (Sprints)

Duration: Weeks 3–10 (4 sprints × 2 weeks each)

Framework: Scrum

Sprint 1 — Authentication & Project Foundation (Weeks 3–4)

Sprint Goal: A working Django project where users can register, log in, and log out.

Task	Assigned	FR	Status
Initialize Django project and app structure	All	—	✓
Implement User model (extend AbstractUser)	Iftekhhar	FR1–FR4	✓
Build registration view and template	Zarin	FR1–FR4	✓
Build login/logout views	Musfiquir	FR5–FR8	✓
Configure URL routing for auth	Towfiq	—	✓
Unit test auth flows	All	FR5, FR6	✓

Sprint 1 Outcome: Users can create accounts, authenticate, and maintain sessions.

Sprint 2 — Product Module & Shopping Cart (Weeks 5–6)

Sprint Goal: Users can browse products and manage their shopping cart.

Task	Assigned	FR	Status
Implement Product model	Iftekhhar	FR12	✓
Product list and detail views/templates	Zarin	FR13–FR16	✓
Implement Cart and CartItem models	Musfiquir	FR17–FR21	✓
Add to Cart / Remove / Update Quantity	Towfiq	FR17–FR19	✓
Dynamic total price calculation	Musfiquir	FR20	✓
Session-based cart persistence	Iftekhhar	FR21	✓
Unit test cart operations	All	FR17–FR21	✓

Sprint 2 Outcome: Full product browsing and cart management operational.

Sprint 3 — Checkout, Orders & Admin Panel (Weeks 7–8)

Sprint Goal: Users can place orders; admins can manage products.

Task	Assigned	FR	Status
Implement Order and OrderItem models	Towfiq	FR25	✓
Checkout view with order confirmation	Zarin	FR22, FR23	✓
Stock decrement on order placement	Iftekhhar	FR24	✓
Order history and detail views	Musfiquir	FR26–FR28	✓
Admin: Add/Edit/Delete product views	All	FR9–FR11	✓
Restrict admin views to staff users	Towfiq	FR9–FR11	✓
Unit test order creation and admin CRUD	All	FR22–FR28	✓

Sprint 3 Outcome: End-to-end shopping flow complete; admin panel functional.

Sprint 4 — UI Polish, Security & Non-Functional Requirements (Weeks 9–10)

Sprint Goal: Application meets all non-functional requirements and is deployment-ready.

Task	Assigned	NFR	Status
Responsive CSS layout (mobile-friendly)	Zarin	NFR Usability	✓
Navigation bar (Home, Cart, Orders, Login)	Musfiqu	NFR Usability	✓
CSRF protection verification	Iftekhar	NFR Security	✓
Password hashing confirmation	Towfiq	NFR Security	✓
Error message display improvements	Zarin	NFR Usability	✓
Code documentation and comments	All	NFR Maintainability	✓
Performance baseline testing	All	NFR Performance	✓
Final integration testing	All	All	✓

Sprint 4 Outcome: Fully polished, secure, and documented application.

4.4 Phase 4 — System Testing & Integration

Duration: Week 11

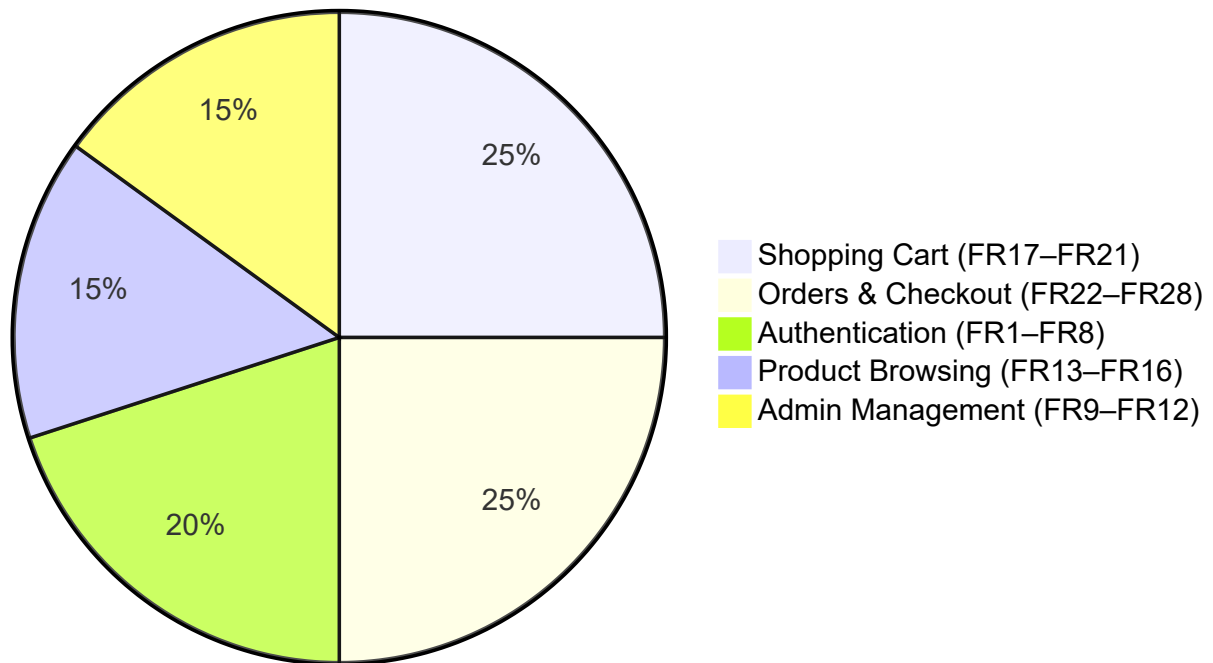
Deliverables: Test Report

Testing Strategies Applied:

Test Type	Scope	Tools
Unit Testing	Individual views, models, and form validation	Django TestCase
Integration Testing	Cart → Checkout → Order flow end-to-end	Django test client
Functional Testing	All FR1–FR28 verified against SRS	Manual + automated
Security Testing	CSRF tokens, session handling, admin access control	Manual
Usability Testing	UI responsiveness across browser sizes	Browser DevTools

Test Coverage Targets:

Test Coverage by Module



4.5 Phase 5 — Deployment

Duration: Week 12

Deliverables: Live application (local or cloud hosted)

Deployment Steps:

1. Finalize `settings.py` for production (`DEBUG=False` , static files configured)
2. Collect static files (`python manage.py collectstatic`)
3. Run database migrations (`python manage.py migrate`)
4. Create superuser account for admin access
5. Deploy locally (development server) or to cloud (PythonAnywhere / Heroku)
6. Perform smoke testing on deployed instance

4.6 Phase 6 — Maintenance & Future Enhancements

Duration: Post-submission / next iteration

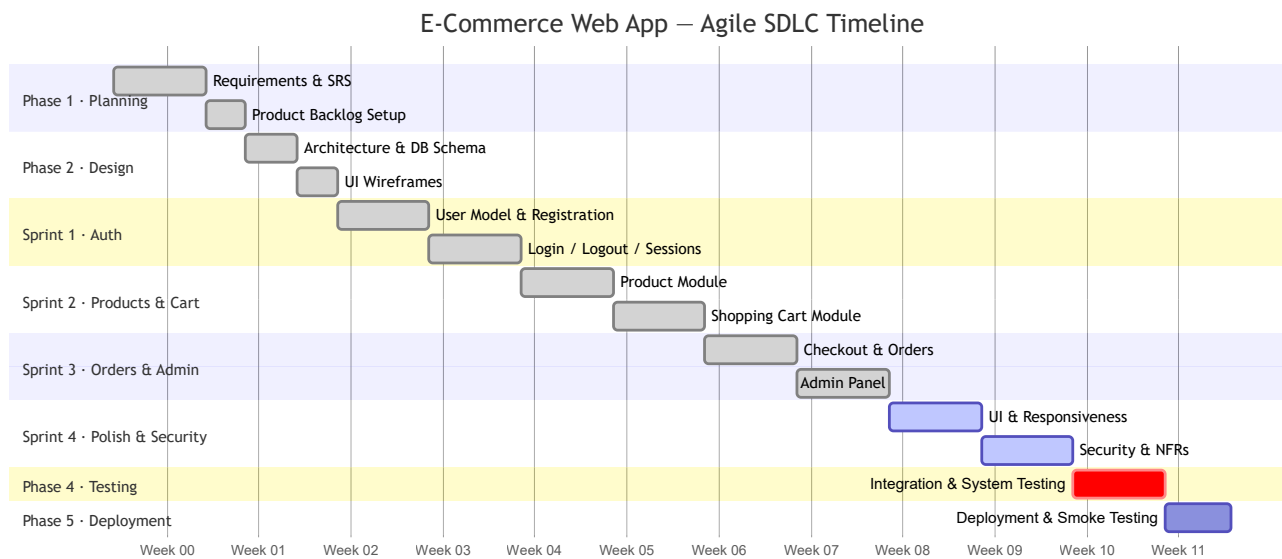
Deliverables: Updated backlog with enhancement items

Backlog items deferred from the current scope (as noted in SRS Section 7):

Enhancement	Priority	Estimated Effort
Online payment integration (Stripe/PayPal)	High	2 sprints
Product search and filtering	High	1 sprint
Product reviews and star ratings	Medium	1 sprint

Enhancement	Priority	Estimated Effort
Wishlist functionality	Medium	1 sprint
Email notifications (order confirmation)	Medium	1 sprint
Admin dashboard analytics	Low	1 sprint

5. Project Timeline (Gantt View)



6. Why Not Waterfall?

Although an SRS document has been written (which is traditionally a Waterfall artifact), the team still benefits from Agile because:

1. **Testing occurs within each sprint**, not only at the end — catching Django ORM bugs, template errors, and routing issues early.
2. **The SRS can evolve** — requirements like pagination, stock display, and admin restrictions were refined during planning, which Waterfall would not accommodate.
3. **Parallel workstreams** — multiple team members can work on separate Django apps simultaneously (products, cart, auth) within a sprint, rather than following a rigid sequential handoff.
4. **Academic peer review** is more natural in a sprint review format, giving instructors and peers regular visibility into incremental progress.

7. Conclusion

The **Agile Scrum-based Iterative SDLC model** is the most appropriate choice for this E-Commerce Web Application. It aligns with the team size of four, the semester timeline, the modular Django architecture, and the iterative nature of web development. The four-sprint plan maps all 28 functional

requirements from the SRS into concrete, deliverable increments — with each sprint producing a working, testable slice of the final product.

This model ensures the team can adapt, collaborate effectively, and deliver a high-quality e-commerce platform on time.